

UNIVERSIDAD PERUANA
CAYETANO HEREDIA

Entregable N°10

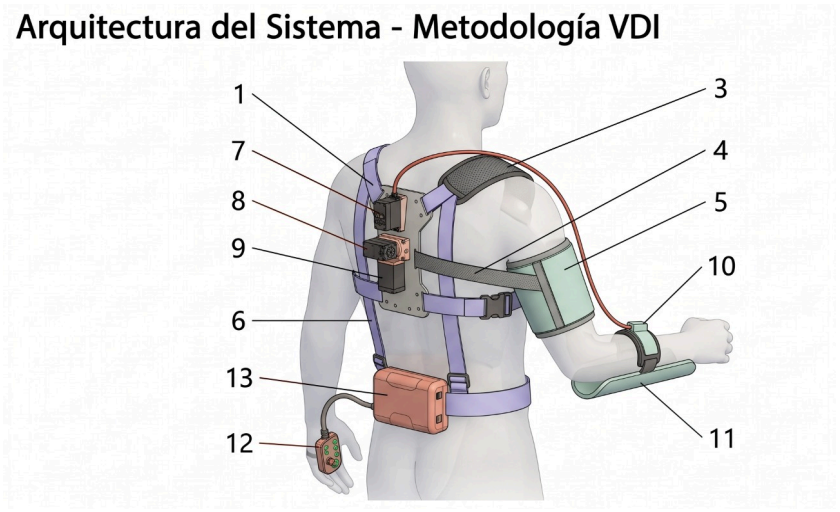
E10: Metodología VDI - Implementación

Caso clínico 4 - Secuela de parálisis braquial obstétrica

Munguia Avila, Ariana Alessandra
Martel Vilca, Dominick Pavel
Monzón La Rosa, María Isabel
Espinoza Cuya, Luis Diego
Guardamino Velasquez, Giovanni Jesus
Huanca Benites, Rodrigo Alonso

Lima – Perú 2026

RESTRUCTURACIÓN DEL BOCETO



Partes del sistema

3. Correa de hombro acolchada
5. Banda del brazo
7. Servo del codo
8. Servo de rotación externa
10. “Ancla del modelaje”: permite conexión
11. Soporte del antebrazo
12. Control remoto
13. Baterías y sistema eléctrico
1-4-6. Sistema de correas.

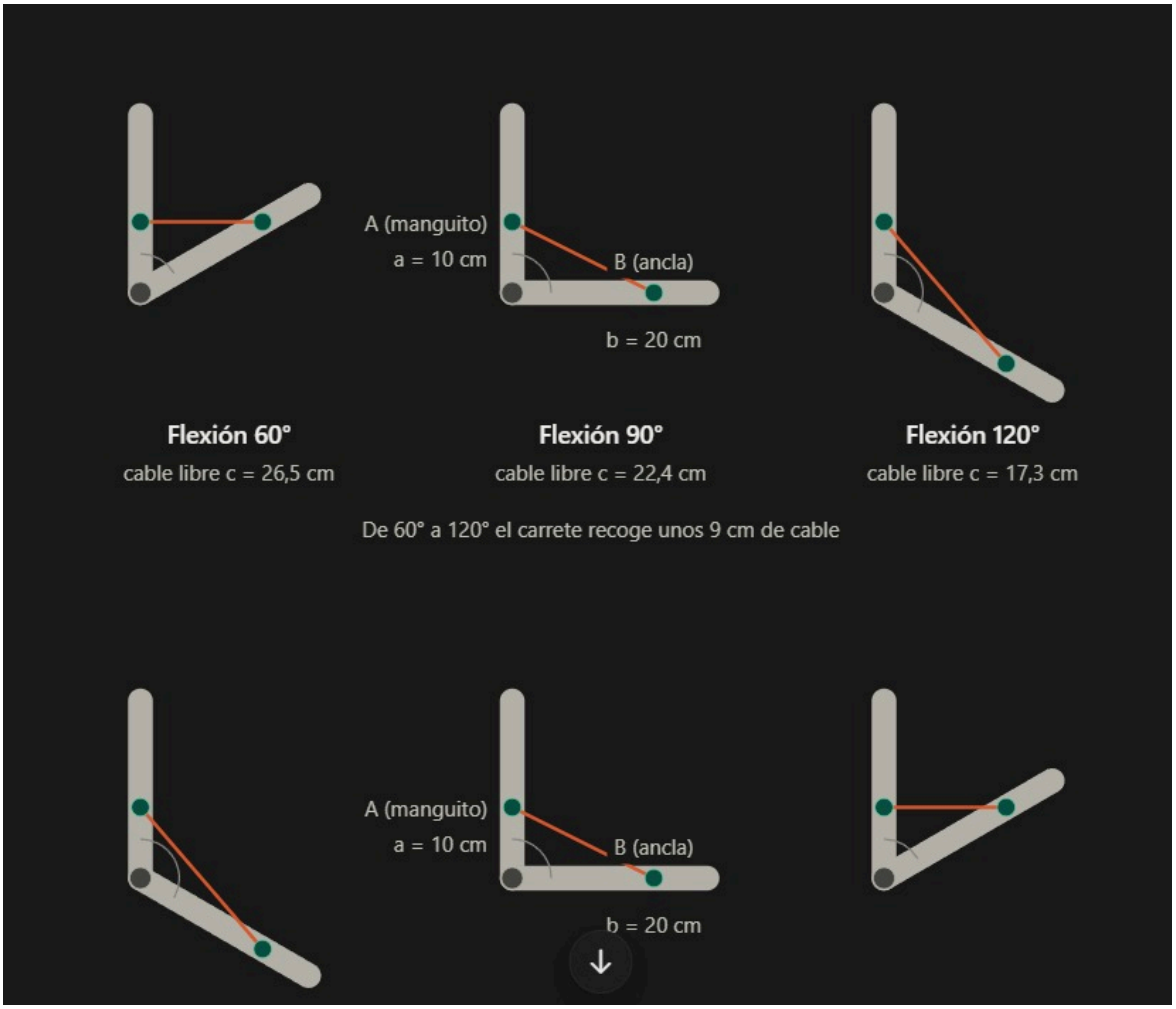


Imagen 1: Movimientos de “modo fundamental” y “rotación del codo”

Para el funcionamiento del prototipo, se llevará a cabo un sistema de correas que permita la correcta movilidad de la órtesis.

Movimientos en relación a rotación del codo:

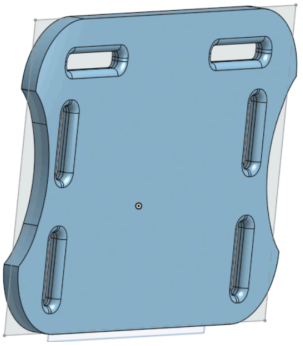
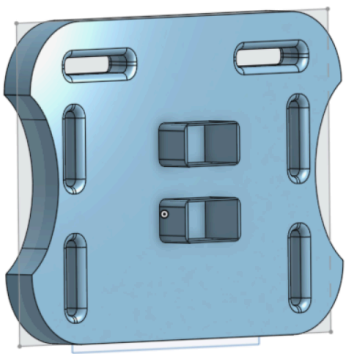
1. El servo gira y el carrete convierte ese giro en la fuerza necesaria para que el soporte del antebrazo suba.
2. El servo enrolla el cable dentro de una funda, permitiendo la elevación del antebrazo; presentando una rotación de codo de 60° a 120° aproximadamente.
3. Para bajar el codo, el servo desenrolla y la gravedad baja el antebrazo, dado que el servo controla la velocidad.

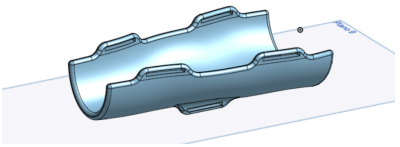
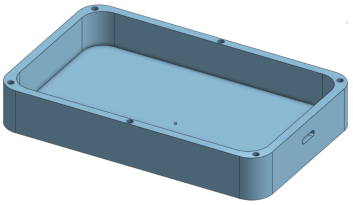
Movimientos en relación a rotación externa del hombro:

4. Otro servo tira de la correa lateral fijada a la banda del brazo, por lo cual el húmero gira y el antebrazo rota de 30 a 45°.

Una observación al respecto es que los cables solo tiran, no empujan el movimiento o la órtesis. Por lo mismo, al “soltar”, la tensión desaparece y el brazo regresa solo.

I. Implementación preliminar de los subsistemas según el diseño

Área	**Implementación esperada**	**Evidencias solicitadas**
Mecánica y fabricación	Piezas fabricadas de acuerdo con los modelos CAD y las especificaciones del diseño.	- Pechera - Parte frontal  - Pechera - Parte Dorsal 

		- Soporte Antebrazo  - Caja de circuitos 
Electrónica y sensores	Circuitos electrónicos montados y conectados según el esquemático	**Foto del montaje e identificación de las partes** ***Área 2: electrónica y sensores*** **Giovanni/Maisa**
Programación y control	Código de programación desarrollado de acuerdo con el diagrama de flujo de control	Respecto al funcionamiento a seguir, se toma de referencia la imagen 3: “Diagrama de lógica de funcionamiento”

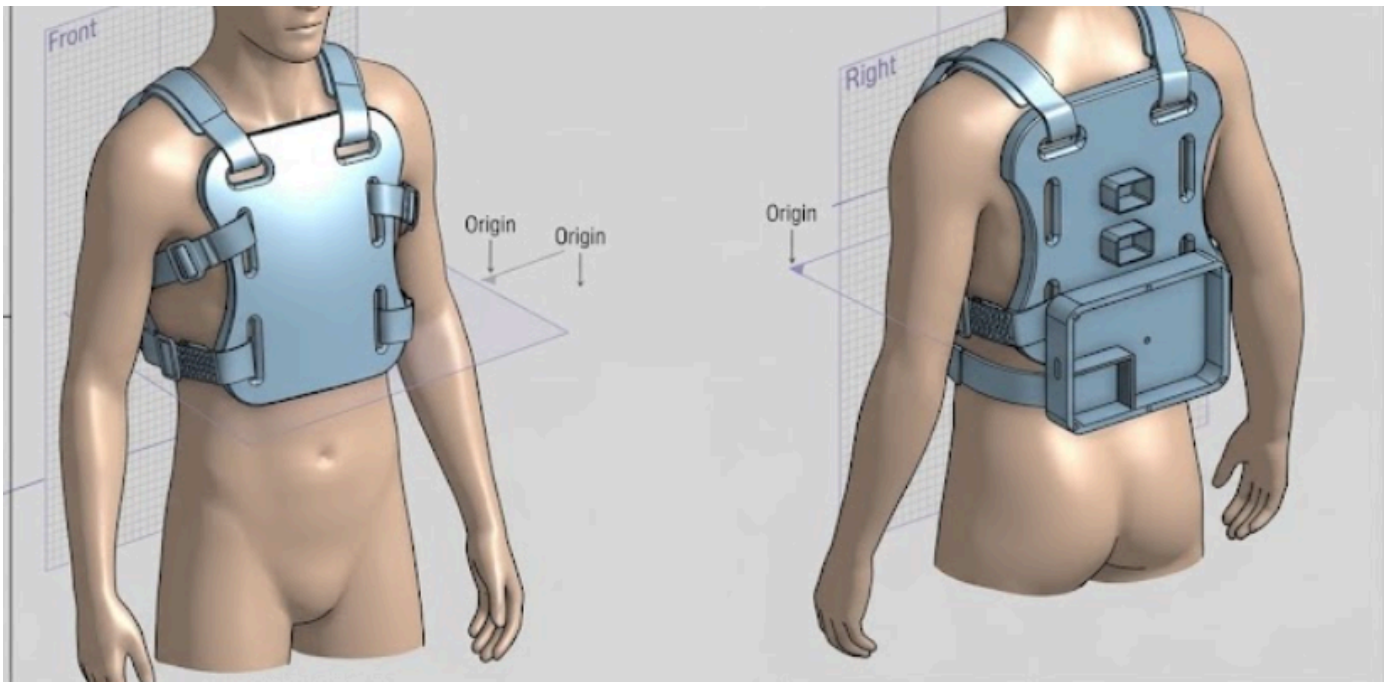


Imagen 2: Modelo 3D - Parte superior ensamblada

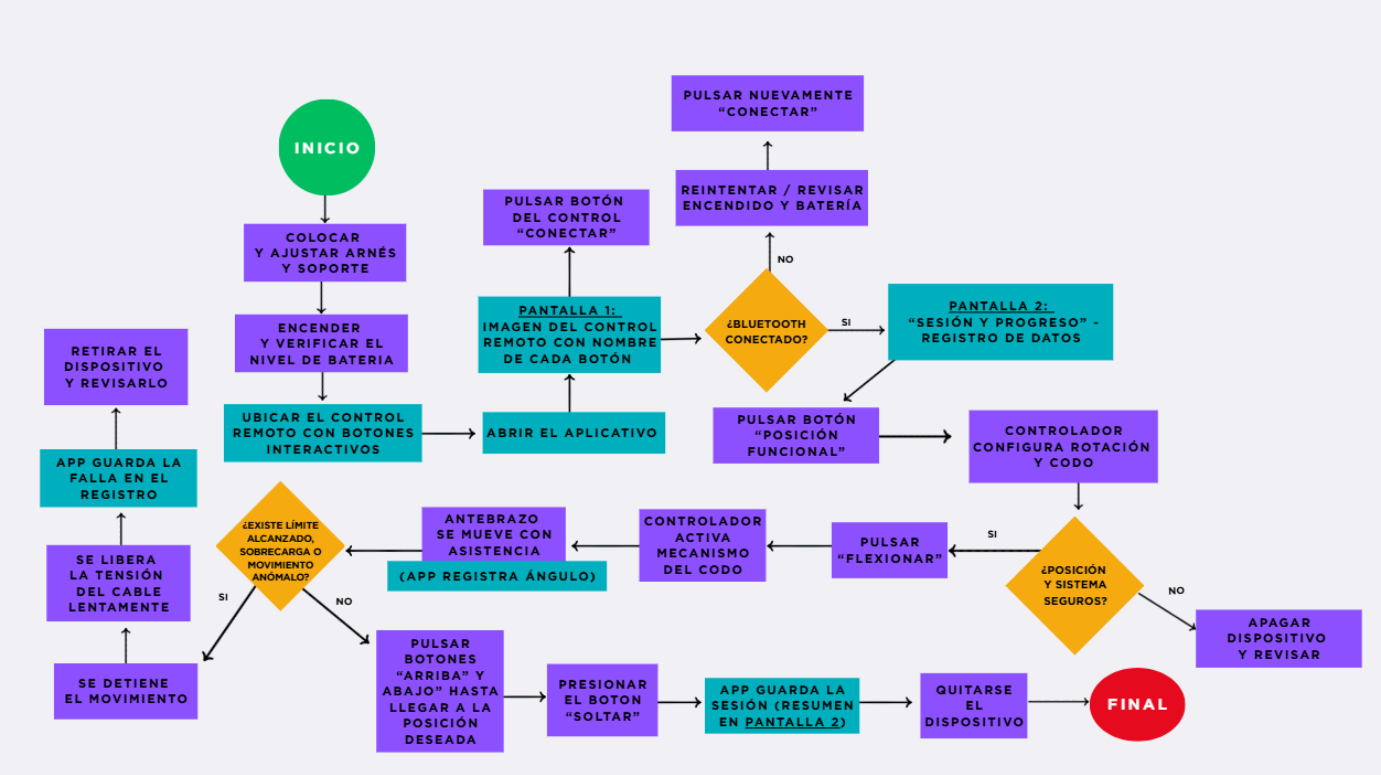


Imagen 3: Diagrama de lógica de funcionamiento

ORTESIS_SERVOS_copy_20261104153844.ino

```
1
2   #include <Arduino.h>
3   #include <Wire.h>
4   #include <math.h>
5   #include <string.h>
6   #include <stdlib.h>
7   #include <ESP32Servo.h>
8   #include <Adafruit_INA219.h>
9   #include <Preferences.h>
10
11  const uint8_t PIN_SERVO_CODO = 19;
12  const uint8_t PIN_SERVO_ROT  = 18;
13  const uint8_t PIN_BTN_POS    = 25;
14  const uint8_t PIN_BTN_UP     = 26;
15  const uint8_t PIN_BTN_DOWN   = 27;
16  const uint8_t PIN_BTN_REL    = 14;
17  const uint8_t PIN_LED        = 13;
18  const uint8_t PIN_BUZZ       = 23;
19  const uint8_t PIN_SDA        = 21;
20  const uint8_t PIN_SCL        = 22;
21
22
23  const float A_CM       = 10.0f;
24  const float B_CM       = 20.0f;
25  const float SPOOL_R_CM = 2.2f;
26
27  const float ELBOW_SERVO_RANGE_DEG = 270.0f;
28  const float ROT_SERVO_RANGE_DEG   = 180.0f;
29  const int   SERVO_US_MIN = 500;
30  const int   SERVO_US_MAX = 2500;
31
32  const float THETA_REST = 50.0f;
33  const float THETA_MIN  = 60.0f;
34  const float THETA_MAX  = 120.0f;
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
35
36  const float ROT_REST_DEG = 0.0f;
37  const float ROT_MAX_DEG  = 90.0f;
38
39  const float ELBOW_SPEED     = 15.0f;
40  const float ELBOW_REL_SPEED = 25.0f;
41  const float ROT_SPEED       = 15.0f;
42  const float ROT_REL_SPEED   = 25.0f;
43
44
45  const float    I_LIMIT_DEFAULT_MA = 2500.0f;
46  const uint16_t OVERCURRENT_MS     = 400;
47  const float    RAIL_MIN_V         = 4.5f;
48  const uint16_t RAIL_LOST_MS       = 300;
49  const bool     ALLOW_NO_INA219    = false;
50
51
52  const uint16_t LOOP_MS     = 20;
53  const uint16_t DEBOUNCE_MS = 30;
54  const uint16_t LONG_MS     = 3000;
55
56
57  enum State : uint8_t { ST_REST, ST_GO_ROT, ST_GO_ELBOW, ST_ASSIST, ST_RELEASING, ST_FAULT };
58  const char* STATE_NAME[] = {"REPOSO", "ROTANDO", "FLEXIONANDO", "ASISTENCIA", "LIBERANDO", "FALLA"};
59
60  Servo servoE, servoR;
61  Adafruit_INA219 ina;
62  Preferences prefs;
63
64  State state = ST_REST;
65  bool servosAttached = false;
66  bool inaOk = false;
67  bool lockout = false;
68  float elbowCur = THETA_REST, elbowTarget = THETA_REST;
69  float rotCur = ROT_REST_DEG, rotTarget = ROT_REST_DEG;
```

```
float thetaFunc = 90.0f;
float rotFunc = 40.0f;
float iLimitMa = I_LIMIT_DEFAULT_MA;

float currentMa = 0, railV = 0;
bool railOk = false;
float overMs = 0, railLostMs = 0;
bool railLost = false;
bool atLimit = false, limitBeeped = false;
bool posLongDone = false;
uint32_t lastTick = 0, lastStatus = 0;


struct Btn {
  uint8_t pin;
  bool state, lastRaw, pressedEdge, releasedEdge;
  uint32_t tChange, tPress;
  explicit Btn(uint8_t p)
      : pin(p), state(false), lastRaw(false), pressedEdge(false), releasedEdge(false), tChange(0), tPress(0) {}
};
Btn btnPos(PIN_BTN_POS), btnUp(PIN_BTN_UP), btnDown(PIN_BTN_DOWN), btnRel(PIN_BTN_REL);

void readBtn(Btn &b, uint32_t now) {
  bool raw = (digitalRead(b.pin) == LOW);
  b.pressedEdge = b.releasedEdge = false;
  if (raw != b.lastRaw) { b.lastRaw = raw; b.tChange = now; }
  if ((now - b.tChange) >= DEBOUNCE_MS && raw != b.state) {
    b.state = raw;
    if (raw) { b.pressedEdge = true; b.tPress = now; }
    else     { b.releasedEdge = true; }
  }
}


```

```
102
103
104 struct Beeper { uint8_t left; uint16_t onMs, offMs; bool on; uint32_t t; } bz = {0, 0, 0, false, 0};
105
106 void beep(uint8_t n, uint16_t onMs, uint16_t offMs = 100) {
107   bz.left = n; bz.onMs = onMs; bz.offMs = offMs; bz.on = false; bz.t = 0;
108 }
109
110 void updateBeeper(uint32_t now) {
111   if (bz.on) {
112     if (now - bz.t >= bz.onMs) {
113       digitalWrite(PIN_BUZZ, LOW);
114       bz.on = false; bz.t = now;
115       if (bz.left) bz.left--;
116     }
117   } else if (bz.left) {
118     if (bz.t == 0 || now - bz.t >= bz.offMs) {
119       digitalWrite(PIN_BUZZ, HIGH);
120       bz.on = true; bz.t = now;
121     }
122   }
123 }
124
125
126 void updateLed(uint32_t now) {
127   bool on;
128   if (lockout)                on = (now / 100) % 2;
129   else switch (state) {
130     case ST_REST:    on = (now % 2000) < 60; break;
131     case ST_ASSIST:  on = atLimit ? ((now / 125) % 2) : true; break;
132     case ST_FAULT:   on = (now / 100) % 2; break;
133     default:         on = (now / 500) % 2; break;
134   }
135   digitalWrite(PIN_LED, on ? HIGH : LOW);
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
135   digitalWrite(PIN_LED, on ? HIGH : LOW);
136 }
137
138
139 float cableLen(float thetaDeg) {
140   float t = thetaDeg * DEG_TO_RAD;
141   return sqrtf(A_CM * A_CM + B_CM * B_CM + 2.0f * A_CM * B_CM * cosf(t));
142 }
143
144
145 float elbowServoDeg(float theta) {
146   return (cableLen(THETA_REST) - cableLen(theta)) / SPOOL_R_CM * RAD_TO_DEG;
147 }
148
149 void writeServoDeg(Servo &s, float deg, float range) {
150   deg = constrain(deg, 0.0f, range);
151   s.writeMicroseconds(SERVO_US_MIN + (int)(deg / range * (SERVO_US_MAX - SERVO_US_MIN)));
152 }
153
154 void drive() {
155   if (!servosAttached) return;
156   writeServoDeg(servoE, elbowServoDeg(elbowCur), ELBOW_SERVO_RANGE_DEG);
157   writeServoDeg(servoR, rotCur, ROT_SERVO_RANGE_DEG);
158 }
159
160
```

```
160
161 void attachServos() {
162   if (servosAttached) return;
163   servoE.setPeriodHertz(50);
164   servoR.setPeriodHertz(50);
165   servoE.attach(PIN_SERVO_CODO, SERVO_US_MIN, SERVO_US_MAX);
166   servoR.attach(PIN_SERVO_ROT, SERVO_US_MIN, SERVO_US_MAX);
167   elbowCur = elbowTarget = THETA_REST;
168   rotCur = rotTarget = ROT_REST_DEG;
169   servosAttached = true;
170   drive();
171 }
172
173 bool moveToward(float &cur, float target, float speed, float dt) {
174   float d = target - cur, step = speed * dt;
175   if (fabsf(d) <= step) { cur = target; return true; }
176   cur += (d > 0) ? step : -step;
177   return false;
178 }
179
180
181 void saveConfig() {
182   prefs.putFloat("thF", thetaFunc);
183   prefs.putFloat("rotF", rotFunc);
184   prefs.putFloat("iLim", iLimitMa);
185 }
186
187 void showConfig() {
188   Serial.printf("Config: codo funcional=%.1f°  rotación funcional=%.1f°  límite=%.0f mA\n",
189                 thetaFunc, rotFunc, iLimitMa);
190   Serial.printf("Servo codo: reposo 0° -> 60° de flexión = %.0f° -> 120° = %.0f° (rango %.0f°)\n",
191                 elbowServoDeg(THETA_MIN), elbowServoDeg(THETA_MAX), ELBOW_SERVO_RANGE_DEG);
192 }
193
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
193
194 void parseCmd(char *c) {
195   if (!strcmp(c, "show"))               showConfig();
196   else if (!strncmp(c, "fe ", 3))       { thetaFunc = constrain((float)atof(c + 3), THETA_MIN, THETA_MAX); showConfig(); }
197   else if (!strncmp(c, "fr ", 3))       { rotFunc = constrain((float)atof(c + 3), ROT_REST_DEG, ROT_MAX_DEG); showConfig(); }
198   else if (!strncmp(c, "ilim ", 5))     { iLimitMa = constrain((float)atof(c + 5), 200.0f, 3000.0f); showConfig(); }
199   else if (!strcmp(c, "save"))          { saveConfig(); Serial.println("Guardado."); }
200   else Serial.println("Comandos: show | fe <60-120> | fr <0-90> | ilim <mA> | save");
201 }
202
203 void handleSerial() {
204   static char buf[32];
205   static uint8_t n = 0;
206   while (Serial.available()) {
207     char ch = (char)Serial.read();
208     if (ch == '\n' || ch == '\r') {
209       if (n) { buf[n] = 0; parseCmd(buf); n = 0; }
210     } else if (n < sizeof(buf) - 1) {
211       buf[n++] = ch;
212     }
213   }
214 }
215
216
217 void enterFault(const char *why) {
218   state = ST_FAULT;
219   elbowTarget = THETA_REST;
220   rotTarget = ROT_REST_DEG;
221   overMs = 0;
222   beep(3, 300, 150);
223   Serial.printf("FALLA: %s (%.0f mA, riel %.2f V). Retrocediendo a reposo; pulsa S4 para salir.\n",
224                 why, currentMa, railV);
225 }
226
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
}

void measure(float dt) {
  float dtMs = dt * 1000.0f;
  if (inaOk) {
    currentMa = ina.getCurrent_mA();
    railV = ina.getBusVoltage_V();
  } else {
    currentMa = 0;
    railV = ALLOW_NO_INA219 ? 6.0f : 0.0f;
  }
  railOk = (railV >= RAIL_MIN_V);
  if (!servosAttached || !inaOk) return;


  if (!railOk) {
    railLostMs += dtMs;
    if (railLostMs >= RAIL_LOST_MS && !railLost) {
      railLost = true;
      state = ST_REST;
      elbowCur = elbowTarget = THETA_REST;
      rotCur = rotTarget = ROT_REST_DEG;
      beep(2, 150, 120);
      Serial.println("Riel de servos caído: reposo. Esperando que vuelva la alimentación.");
    }
  } else {
    railLostMs = 0;
    railLost = false;
  }


  if (railOk && fabsf(currentMa) > iLimitMa) overMs += dtMs;
  else overMs = fmaxf(0.0f, overMs - 2.0f * dtMs);
  if (overMs >= OVERCURRENT_MS && state != ST_FAULT) enterFault("sobrecorriente");
}
```

```
260
261 void requestFunctional() {
262   if (lockout || state == ST_FAULT || !railOk) { beep(2, 60, 60); return; }
263   attachServos();
264   state = ST_GO_ROT;
265 }
266
267 void requestRelease() {
268   if (!servosAttached || state == ST_REST) return;
269   state = ST_RELEASING;
270 }
271
272 void handleButtons(uint32_t now) {
273   if (lockout) return;
274
275   if (btnRel.pressedEdge) {
276     if (state == ST_FAULT) {
277       bool atRest = (elbowCur == THETA_REST && rotCur == ROT_REST_DEG);
278       if (atRest && fabsf(currentMa) < iLimitMa) { state = ST_REST; beep(1, 80); }
279     } else {
280       requestRelease();
281     }
282   }
283
284   if (btnPos.pressedEdge) posLongDone = false;
285   if (btnPos.state && !posLongDone && state == ST_ASSIST && (now - btnPos.tPress) >= LONG_MS) {
286     posLongDone = true;
287     thetaFunc = constrain(elbowCur, THETA_MIN, THETA_MAX);
288     saveConfig();
289     beep(3, 60, 60);
290     Serial.printf("Posición funcional guardada: %.1f°\n", thetaFunc);
291   }
292   if (btnPos.releasedEdge && !posLongDone && (now - btnPos.tPress) < LONG_MS) requestFunctional();
293 }
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
293  }
294
295  void runState(float dt) {
296    float eSp = ELBOW_SPEED, rSp = ROT_SPEED;
297    switch (state) {
298      case ST_GO_ROT:   rotTarget = rotFunc; break;
299      case ST_GO_ELBOW: elbowTarget = thetaFunc; break;
300      case ST_ASSIST:
301        if (btnUp.state && !btnDown.state)      elbowTarget = THETA_MAX;
302        else if (btnDown.state && !btnUp.state) elbowTarget = THETA_MIN;
303        else                                    elbowTarget = elbowCur;
304        break;
305      case ST_RELEASING:
306      case ST_FAULT:
307        elbowTarget = THETA_REST; rotTarget = ROT_REST_DEG;
308        eSp = ELBOW_REL_SPEED; rSp = ROT_REL_SPEED;
309        break;
310      default: break;
311    }
312
313    bool eDone = moveToward(elbowCur, elbowTarget, eSp, dt);
314    bool rDone = moveToward(rotCur, rotTarget, rSp, dt);
315
316    if (state == ST_GO_ROT && rDone)                    state = ST_GO_ELBOW;
317    else if (state == ST_GO_ELBOW && eDone)             { state = ST_ASSIST; beep(1, 60); }
318    else if (state == ST_RELEASING && eDone && rDone)   state = ST_REST;
319
320
321    atLimit = (state == ST_ASSIST) &&
322              ((btnUp.state && !btnDown.state && elbowCur >= THETA_MAX - 0.01f) ||
323               (btnDown.state && !btnUp.state && elbowCur <= THETA_MIN + 0.01f));
324    if (atLimit && !limitBeeped) { beep(1, 40); limitBeeped = true; }
325    if (!btnUp.state && !btnDown.state) limitBeeped = false;
326  }
327
```

ORTESIS_SERVOS_copy_20261104153844.ino

```
328
329 void setup() {
330   Serial.begin(115200);
331   pinMode(PIN_BTN_POS, INPUT_PULLUP);
332   pinMode(PIN_BTN_UP, INPUT_PULLUP);
333   pinMode(PIN_BTN_DOWN, INPUT_PULLUP);
334   pinMode(PIN_BTN_REL, INPUT_PULLUP);
335   pinMode(PIN_LED, OUTPUT);
336   pinMode(PIN_BUZZ, OUTPUT);
337   digitalWrite(PIN_BUZZ, LOW);
338
339   ESP32PWM::allocateTimer(0);
340   ESP32PWM::allocateTimer(1);
341
342   Wire.begin(PIN_SDA, PIN_SCL);
343   inaOk = ina.begin();
344   if (inaOk) ina.setCalibration_32V_2A();
345
346   prefs.begin("ortesis", false);
347   thetaFunc = constrain(prefs.getFloat("thF", 90.0f), THETA_MIN, THETA_MAX);
348   rotFunc   = constrain(prefs.getFloat("rotF", 40.0f), ROT_REST_DEG, ROT_MAX_DEG);
349   iLimitMa  = prefs.getFloat("iLim", I_LIMIT_DEFAULT_MA);
350
351   Serial.println("\nÓrtesis con servos - ESP32");
352   showConfig();
353   if (elbowServoDeg(THETA_MAX) > ELBOW_SERVO_RANGE_DEG) {
354     lockout = true;
355     Serial.println("ERROR: el carrete necesita más giro del que da el servo del codo. Aumenta SPOOL_R_CM o usa un servo de más rango.");
356   }
357   if (!inaOk && !ALLOW_NO_INA219) {
358     lockout = true;
359     Serial.println("ERROR: no se encuentra el INA219 (I2C 0x40). Servos deshabilitados.");
360   }
361   beep(lockout ? 3 : 1, 100, 100);
362 }
362   }
363
364   void loop() {
365     uint32_t now = millis();
366     updateBeeper(now);
367     updateLed(now);
368     handleSerial();
369
370     if (now - lastTick < LOOP_MS) return;
371     float dt = (now - lastTick) / 1000.0f;
372     lastTick = now;
373
374     readBtn(btnPos, now); readBtn(btnUp, now); readBtn(btnDown, now); readBtn(btnRel, now);
375     measure(dt);
376     handleButtons(now);
377     runState(dt);
378     drive();
379
380     if (now - lastStatus >= 1000) {
381       lastStatus = now;
382       Serial.printf("[%s] codo=%.1f° rot=%.0f° I=%.0f mA riel=%.2f V\n",
383                     STATE_NAME[state], elbowCur, rotCur, currentMa, railV);
384     }
385   }
386
```

Imagen 4: Programación

II. Verificación preliminar del diseño por subsistemas

Área	Tipo de verificación	Evidencias esperadas
Mecánica y fabricación	Verificación dimensional y funcional de las piezas.	Tipos de verificación de dimensiones 1. Verificación de los tamaños de piezas 3D (imagen 5 e imagen 6) 2. Ajuste las piezas impresas encajen entre ellas y no sea incómodo para la usuaria. 3. Prueba de elementos: organizar elementos electrónicos 4. Prueba de movimiento: con los elementos ensamblados y con correas, realizar los movimientos. Las evidencias que buscamos son: ● Foto de ajustes de pieza y del cable en la ranura. ● Registro del peso aproximado ● Recorrido real frente al calculado/esperado (considerando ángulos) ***Área 1: mecánica y fabricación*** **1. Impresión de piezas CAD y ajuste entre piezas** ● **Fabricación de componentes CAD:** Se fabricaron las piezas estructurales de la órtesis utilizando manufactura aditiva FDM con filamentos resistentes (PETG y PLA+), junto con almohadillas anatómicas en TPU flexible para prevenir irritación cutánea. Las piezas fabricadas de acuerdo con el modelo 3D son: ○ **Pechera - Parte frontal:** Placa rígida de soporte frontal con un tamaño total de 250 x 180 mm. Cuenta con ranuras superiores de 52 x 100 mm para el anclaje de las correas de los hombros y ranuras laterales de 10 x 42 mm para el ajuste del torso. ○ **Pechera - Parte dorsal:** Chasis posterior de 250 x 180 mm que integra ranuras de sujeción (52 x 10 mm y 10 x 42 mm) e incluye bloques de soporte rectangulares de 44 x 24 mm sobresalientes para alojar y fijar los servomotores del sistema. ○ **Soporte Antebrazo:** Estructura cóncava ergonómica con dimensiones generales de (19.5 x 92.6 x 33.5 mm). Incorpora ranuras de 27.5 x 2.75 mm orientadas al guiado y paso del cableado de asistencia. ○ **Caja de circuitos:** Carcasa protectora (160 x 100 x 2.5 mm)de dos piezas destinada a

albergar la placa de control (PCB), la batería y el módulo de carga USB-C.

- **Ajuste entre piezas:** Se verificó experimentalmente que el ensamblaje mantenga una holgura adecuada en las ranuras para permitir el paso fluido de las cinchas de velcro, así como un encaje firme en los compartimentos de la caja de circuitos y servomotores para prevenir vibraciones o desalineaciones durante el funcionamiento dinámico.

2. **Requisitos de tolerancia aceptada:**

    - **Tolerancia Dimensional General:** Se fijó y verificó una tolerancia de manufactura de $\pm$ 0.15 mm a $\pm$0.20 mm en la impresión 3D FDM. Esto garantiza que las dimensiones reales de los soportes y ejes encajen sin juego holgado ni aprisionamiento sobre los rodamientos.

    - **Tolerancia en Roscas e Insertos:** Las cavidades destinadas a la inserción térmica de tuercas de latón (M2.5 y M3 a 220°C) se diseñaron con una tolerancia de interferencia ligera para evitar que se desprendan frente al torque de los motores.

    - **Tolerancia con Holgura (Juego Mecánico):** Se mantuvo una tolerancia abierta en el perfil interior de las ranuras de guiado de correas para absorber variaciones en el grosor del textil o del velcro sin trabar el ajuste ergonómico sobre la paciente.

Topes Mecánicos

A. Descripción y Función Biomecánica

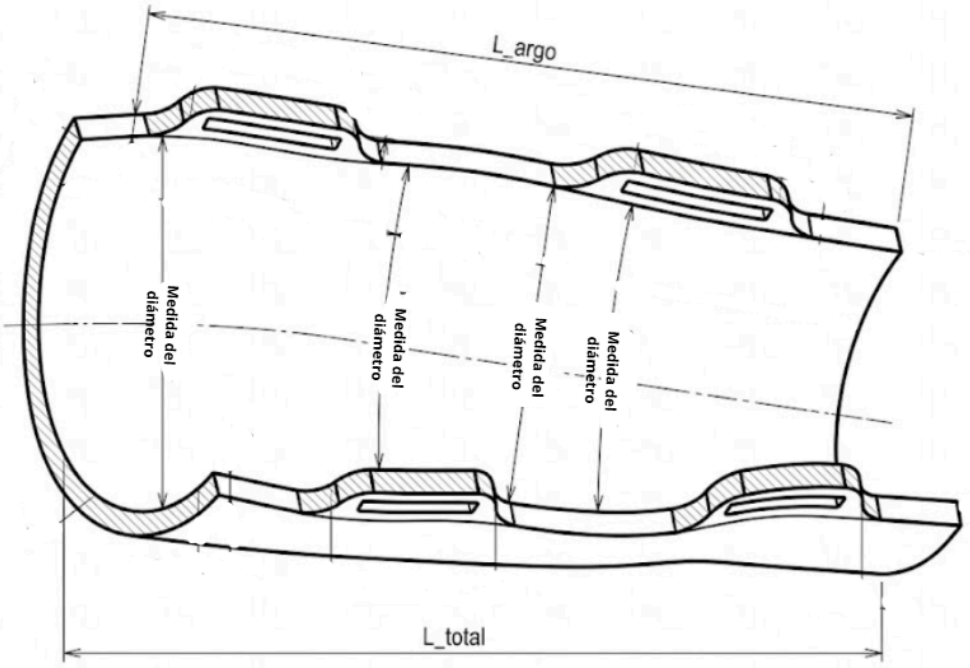
- **Propósito:** Prevenir subluxaciones glenohumerales, tirones musculoesqueléticos o lesiones secundarias derivadas de la parálisis braquial obstétrica (PBO) mediante la restricción física de la excursión angular.
- **Límites Angulares Restringidos:**
    - **Abducción del Hombro:** Restringida mecánicamente a un tope máximo de 80° (ajustable en evaluación clínica inicial hasta 40°).
    - **Rotación Externa del Hombro:** Delimitada físicamente a un ángulo máximo de 45° (o 30° según la necesidad clínica).

		B. Implementación y Verificación Práctica ● **Implementación Física:** Se diseñaron e imprimieron bloqueadores rígidos y pasadores fijos colocados sobre los ejes articulados de la estructura (PETG/PLA+), actuando como tope de seguridad en caso de fallo electrónico, error de la IMU o pérdida del enlace inalámbrico. ● **Método de Verificación:** Se aplicaron cargas estáticas y fuerza manual directa contra los topes impresos, evaluando con un goniómetro directo que la articulación no sobrepase los ángulos límite prefijados bajo la aplicación del torque nominal del servomotor.
Electrónica y sensores	Verificación de conexiones eléctricas y funcionamiento individual de sensores y actuadores.	**1. Verificación de voltaje:** ● Comprueba que la alimentación eléctrica y las conexiones de corriente estén estables y dentro de los rangos seguros para los componentes **2. Verificación de sensores:** ● Validar que el microcontrolador lea correctamente las señales analógicas y digitales **3. Verificación de actuadores:** ● Confirmar que los servomotores y mecanismos responda de manera física y precisa a las señales de salida **4. Verificación de conexión y pantallas:** ● Comprobar la vinculación vía Bluetooth y la respuesta del aplicativo (Control y de registro de datos) **5. Verificación de Entradas:** ● Verificar la correcta lectura de las señales de los botones - “Arriba”, “Abajo”, “Soltar” y “Posición funcional” **6. Control de Actuadores:** ● Comprobar la ejecución de la rotación y el movimiento asistido del codo mediante los servomotores según el comando enviado

		7. Prueba de lógica de seguridad y sobrecarga: ● Simular una lectura del sensor de tensión superior al límite establecido para comprobar que el sistema detiene automáticamente el movimiento, libera la tensión de manera lenta y guarda la falla en el registro de la aplicación
Programación y control	Verificación de lógica de programación y respuesta de entradas y salidas.	1. Entradas del sistema (Inputs) - **Conexión Bluetooth:** ● Estado de vinculación ("¿Bluetooth conectado?" -> Sí / No). - **Comandos de Botones del Aplicativo:** ● Pulsar botón "Conectar" ● Pulsar el botón "Posición funcional". ● Pulsar "Flexionar". ● Pulsación de botones "Arriba" y "Abajo". ● Presionar el botón "Soltar" - **Sensores de Estado:** ● Verificación de seguridad inicial ("¿Posición y sistema seguros?" -> Sí / No). ● Verificación de límites ("¿Existe límite alcanzado, sobrecarga o movimiento anómalo?" -> Sí / No). 2. Salida del sistema (Outputs) - **Pantallas y Registros de la App:** ● Despliegue de Pantalla 1 (Imagen del control remoto con nombre de cada botón). ● Despliegue de Pantalla 2 ("Sesión y progreso"). ● La App registra el ángulo de movimiento. ● La App guarda la falla en el registro.

		● La App guarda la sesión (resumen en Pantalla 2). - **Acciones de Control de actuadores:** ● Controlador configura rotación y codo. ● Controlador activa mecanismo del Antebrazo se mueve con asistencia. ● Se detiene el movimiento. ● Se libera la tensión del cable lentamente. ● Apagar el dispositivo. **Área: Programación y control** **A. Código Implementado y Lógica de Funcionamiento** **1. Implementación esperada** ● **Arquitectura de Procesamiento:** El programa está desarrollado para un microcontrolador ESP32 DevKit. Regula el lazo de control, lee los comandos enviados desde la interfaz inalámbrica o botonera y monitorea las variables inerciales y de corriente. ● **Lógica del Algoritmo de Control:** ○ **Inicialización y Vinculación:** Sincroniza la comunicación Bluetooth, realiza la lectura de cero inicial de la IMU (MPU-6050) y evalúa los estados de batería. ○ **:Control de Asistencia:** Al recibir comandos de movimiento ("Posición funcional", "Flexionar" "Extender"), el microcontrolador genera señales de Modulación por Ancho de Pulso (PWM) dirigidas a los servomotores de codo (35 kg . cm) y rotación (20 kg . cm alimentados a 6V. ○ **Protección Activa:** Si el software detecta un ángulo fuera del rango seguro o un consumo anómalo de corriente, suspende inmediatamente

		el pulso PWM al motor, detiene el movimiento y libera gradualmente la tensión del cable.



Medida	Diámetro (cm)	Contorno (cm)
1	6,5	23
2	6,5	23
3	6,5	21,5
4	5,5	20,5
5	5,5	19,5
6	5,7	17,5
7	5,5	16.8
8	5,5	15,7

Medida	Largo del antebrazo (cm)
1	23cm

Imagen 5: Dimensiones de la órtesis en antebrazo